

Instituto Tecnológico Superior de Lerdo

Ingeniería Informática

Programación Orientada a Objetos



Nombre del alumno: Emiliano Gonzalez Liserio

No. Control: 252310156

Actividad:

Actividad de evaluacion c3 - usuarios

Lerdo, Durango 10/05/2026

Evidencia de ejecución

```
PS E:\XAMPP_webs\unidad_4\Act_evaluacion> & C:/Users/emili/AppData/Local/Programs/Python/Python313/p

LISTADO DE USUARIOS DEL SISTEMA

► Usuario #1 (Admin)
¡Hola, Carlos Ramírez! Bienvenido al sistema.
Nombre : Carlos Ramírez
Email : carlos@admin.com
Nivel de acceso: 5
[Carlos Ramírez] ADMINISTRADOR – Acceso TOTAL. Nivel: 5

► Usuario #2 (Cliente)
¡Hola, Ana Torres! Bienvenido al sistema.
Nombre : Ana Torres
Email : ana@cliente.com
Puntos acumulados: 350
[Ana Torres] CLIENTE – Acceso al catálogo y perfil personal. Puntos: 350

► Usuario #3 (Invitado)
¡Hola, Luis Gómez! Bienvenido al sistema.
Nombre : Luis Gómez
Email : luis@correo.com
[Luis Gómez] INVITADO – Solo lectura. Acceso limitado a contenido público.

SISTEMA DE USUARIOS v1.0
1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 1

LISTADO DE USUARIOS DEL SISTEMA

► Usuario #1 (Admin)
¡Hola, Carlos Ramírez! Bienvenido al sistema.
Nombre : Carlos Ramírez
Email : carlos@admin.com
Nivel de acceso: 5
```

LISTADO DE USUARIOS DEL SISTEMA

► Usuario #1 (Admin)

¡Hola, Carlos Ramírez! Bienvenido al sistema.

Nombre : Carlos Ramírez

Email : carlos@admin.com

Nivel de acceso: 5

[Carlos Ramírez] ADMINISTRADOR – Acceso TOTAL. Nivel: 5

► Usuario #2 (Cliente)

¡Hola, Ana Torres! Bienvenido al sistema.

Nombre : Ana Torres

Email : ana@cliente.com

Puntos acumulados: 350

[Ana Torres] CLIENTE – Acceso al catálogo y perfil personal. Puntos: 350

► Usuario #3 (Invitado)

¡Hola, Luis Gómez! Bienvenido al sistema.

Nombre : Luis Gómez

Email : luis@correo.com

[Luis Gómez] INVITADO – Solo lectura. Acceso limitado a contenido público.

SISTEMA DE USUARIOS v1.0

- | |
|--|
| <ol style="list-style-type: none">1. Mostrar todos los usuarios2. Agregar Administrador3. Agregar Cliente4. Agregar Invitado5. Salir |
|--|

Elige una opción: 2

Nombre del admin: Emiliano Gonzalez Liserio

Email: emiliano@gmail.com

Nivel de acceso (1-5): 5

✓ Admin 'Emiliano Gonzalez Liserio' registrado.

SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 2

Nombre del admin: Emiliano Gonzalez Liserio

Email: emiliano@gmail.com

Nivel de acceso (1-5): 5

✓ Admin 'Emiliano Gonzalez Liserio' registrado.

SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 3

Nombre del cliente: Emiliano Gonzalez Liserio

Email: clienteem@gmail.com

Puntos iniciales (0): 0

✓ Cliente 'Emiliano Gonzalez Liserio' registrado.

SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 4

✓ Cliente 'Emiliano Gonzalez Liserio' registrado.

SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 4

Nombre del invitado: Iron Man

Email: ironman@gmail.com

✓ Invitado 'Iron Man ' registrado.

SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 1

LISTADO DE USUARIOS DEL SISTEMA

- ▶ Usuario #1 (Admin)
¡Hola, Carlos Ramírez! Bienvenido al sistema.
Nombre : Carlos Ramírez
Email : carlos@admin.com
Nivel de acceso: 5
[Carlos Ramírez] ADMINISTRADOR – Acceso TOTAL. Nivel: 5
- ▶ Usuario #2 (Cliente)
¡Hola, Ana Torres! Bienvenido al sistema.
Nombre : Ana Torres
Email : ana@cliente.com
Puntos acumulados: 350
[Ana Torres] CLIENTE – Acceso al catálogo y perfil personal. Puntos: 350

LISTADO DE USUARIOS DEL SISTEMA

- ▶ Usuario #1 (Admin)
¡Hola, Carlos Ramírez! Bienvenido al sistema.
Nombre : Carlos Ramírez
Email : carlos@admin.com
Nivel de acceso: 5
[Carlos Ramírez] ADMINISTRADOR – Acceso TOTAL. Nivel: 5

- ▶ Usuario #2 (Cliente)
¡Hola, Ana Torres! Bienvenido al sistema.
Nombre : Ana Torres
Email : ana@cliente.com
Puntos acumulados: 350
[Ana Torres] CLIENTE – Acceso al catálogo y perfil personal. Puntos: 350

- ▶ Usuario #3 (Invitado)
¡Hola, Luis Gómez! Bienvenido al sistema.
Nombre : Luis Gómez
Email : luis@correo.com
[Luis Gómez] INVITADO – Solo lectura. Acceso limitado a contenido público.

- ▶ Usuario #4 (Admin)
¡Hola, Emiliano Gonzalez Liserio! Bienvenido al sistema.
Nombre : Emiliano Gonzalez Liserio
Email : emiliano@gmail.com
Nivel de acceso: 5
[Emiliano Gonzalez Liserio] ADMINISTRADOR – Acceso TOTAL. Nivel: 5

- ▶ Usuario #5 (Cliente)
¡Hola, Emiliano Gonzalez Liserio! Bienvenido al sistema.
Nombre : Emiliano Gonzalez Liserio
Email : clienteem@gmail.com
Puntos acumulados: 0
[Emiliano Gonzalez Liserio] CLIENTE – Acceso al catálogo y perfil personal. Puntos: 0

- ▶ Usuario #6 (Invitado)
¡Hola, Iron Man ! Bienvenido al sistema.
Nombre : Iron Man
Email : ironman@gmail.com
[Iron Man] INVITADO – Solo lectura. Acceso limitado a contenido público.

SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 5

Cerrando sistema. ¡Hasta pronto!

Todas estas son la pruebas que se pueden hacer

Ahora control de errores

```
PS E:\XAMPP_webs\unidad 4\Act_evaluacion> & C:/Users/emili/AppData/Local/Programs/Python/Python38-64/Python.exe E:\XAMPP_webs\unidad 4\Act_evaluacion\usuarios.py

=====
LISTADO DE USUARIOS DEL SISTEMA
=====

▶ Usuario #1 (Admin)
¡Hola, Carlos Ramírez! Bienvenido al sistema.
Nombre : Carlos Ramírez
Email : carlos@admin.com
Nivel de acceso: 5
[Carlos Ramírez] ADMINISTRADOR – Acceso TOTAL. Nivel: 5

▶ Usuario #2 (Cliente)
¡Hola, Ana Torres! Bienvenido al sistema.
Nombre : Ana Torres
Email : ana@cliente.com
Puntos acumulados: 350
[Ana Torres] CLIENTE – Acceso al catálogo y perfil personal. Puntos: 350

▶ Usuario #3 (Invitado)
¡Hola, Luis Gómez! Bienvenido al sistema.
Nombre : Luis Gómez
Email : luis@correo.com
[Luis Gómez] INVITADO – Solo lectura. Acceso limitado a contenido público.
=====



SISTEMA DE USUARIOS v1.0
    

1. Mostrar todos los usuarios  

        2. Agregar Administrador  

        3. Agregar Cliente  

        4. Agregar Invitado  

        5. Salir



Elige una opción: 2
Nombre del admin: Emiliano Gonzalez Liserio
Email: emilianogonzalez
Nivel de acceso (1-5): 5
✗Error: Email inválido: emilianogonzalez
```

Aqui verifica el email y si no cumple con los criterios lo rechaza

```
SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 2
Nombre del admin: Emiliano Gonzalez
Email: emi@gmail.com
Nivel de acceso (1-5): alto
✗Error: Nivel de acceso inválido: 'alto'. Debe ser un número entero, no texto.
```

Aqui se verifica que el input de nivel este dentro del rango y sea int no string

```
SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 3
Nombre del cliente: Emiliano Gonzalez
Email: emi@gmail.com
Puntos iniciales (0): -200
✗Error: Los puntos no pueden ser negativos: -200.
```

```
SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: 3
Nombre del cliente: Emiliano Gonzalez
Email: emi@gmail.com
Puntos iniciales (0): docientos
✗Error: Puntos inválidos: 'docientos'. Deben ser un número entero, no texto.
```

Aqui verifica que los puntos sean 0 o positivos tambien que sean numeros y no strings

ABPJ · CORTE 3 · POO

Sistema de Usuarios con Herencia en Python

Aplicación del concepto de herencia y polimorfismo para modelar distintos tipos de usuarios en una plataforma digital.

INSTITUCIÓN

Instituto Tecnológico Superior de Lerdo

MATERIA

Programación Orientada a Objetos

ALUMNO

Emiliano Gonzalez Liserio

2 Explicación del Problema

Una plataforma digital necesita gestionar distintos tipos de usuarios. Todos comparten información básica (nombre y email), pero cada tipo tiene comportamientos y permisos distintos. El sistema usa **herencia** para evitar repetir código: una clase base centraliza lo común y las clases hijas añaden lo específico.

ADMINISTRADOR

- Acceso total al sistema
- Tiene nivel de acceso (1-5)
- Puede gestionar usuarios
- Permisos máximos

CLIENTE

- Acceso a catálogo y perfil
- Acumula puntos de compra
- Puede canjear puntos
- Permisos medios

INVITADO

- Solo lectura pública
- Acceso mínimo
- Sin datos adicionales
- Permisos mínimos

3 Diagrama de Clases – Herencia (UML)



4 Explicación de cada Clase – con validaciones

Usuario – Clase Base

```

nombre, email
mostrar_datos()
acceso_sistema()
saludar()
_validar_email()
formato válido: user@dominio.com
sin @ o dominio → ValueError
  
```

Centraliza la lógica común. El email se valida con regex al construir cualquier tipo de usuario.

Admin – Clase Derivada

```

nivel_acceso propio: int
_validar_nivel() nuevo
acceso_sistema() sobrescrito
gestionar_usuarios()
entero entre 1 y 5
texto como "admin" → ValueError
decimal "3.5" → ValueError
fuera de rango (0, 6, -1) → ValueError
  
```

La validación convierte el valor a int y verifica el rango. Acepta "5" (string) pero rechaza "abc" o 6.

4 Explicación de cada Clase – con validaciones

Usuario – Clase Base

```
nombre, email
mostrar_datos()
acceso_sistema()
saludar()
_validar_email()
formato válido: user@dominio.com
sin @ o dominio → ValueError
```

Centraliza la lógica común. El email se valida con regex al construir cualquier tipo de usuario.

Admin – Clase Derivada

```
nivel_acceso propio · int
_validar_nivel() nuevo
acceso_sistema() sobrescrito
gestionar_usuarios()
entero entre 1 y 5
texto como "admin" → ValueError
decimal "3.5" → ValueError
fuera de rango (0, 6, -1) → ValueError
```

La validación convierte el valor a int y verifica el rango. Acepta "5" (string) pero rechaza "abc" o 6.

Cliente – Clase Derivada

```
puntos propio · int
_validar_puntos() nuevo
acceso_sistema() sobrescrito
canjear_puntos(n)
entero entre 0 y 100,000
texto como "hola" → ValueError
negativo como -10 → ValueError
mayor a 100,001+ → ValueError
```

Invitado – Clase Derivada

Sin atributos adicionales
Sin validaciones propias
`acceso_sistema()` sobrescrito
solo hereda validación de email

La clase más simple. No necesita validaciones adicionales porque no tiene atributos propios más allá de los de Usuario.

Tres mensajes de error distintos según el tipo de fallo: tipo incorrecto, negativo o fuera de límite.

5 Fragmentos de Código – validaciones incluidas

admin.py – _validar_nivel() • mismo patrón que email

```
class Admin(Usuario):
    NIVEL_MIN = 1
    NIVEL_MAX = 5

    def __init__(self, nombre, email, nivel_acceso):
        super().__init__(nombre, email)
        self.nivel_acceso = self._validar_nivel(nivel_acceso)

    def _validar_nivel(self, nivel):
        try:
            nivel_int = int(nivel)          # falla si es "abc" o "3.5"
        except (ValueError, TypeError):
            raise ValueError(
                f"Nivel inválido: '{nivel}'. Debe ser un número entero."
            )
        if not (self.NIVEL_MIN ≤ nivel_int ≤ self.NIVEL_MAX):
            raise ValueError(
                f"Fuera de rango: {nivel_int}. Debe estar entre "
                f"{self.NIVEL_MIN} y {self.NIVEL_MAX}."
            )
        return nivel_int
```

cliente.py – _validar_puntos() • tres casos de error

cliente.py - _validar_puntos() · tres casos de error

```
class Cliente(Usuario):
    PUNTOS_MIN = 0
    PUNTOS_MAX = 100_000

    def __init__(self, nombre, email, puntos=0):
        super().__init__(nombre, email)
        self.puntos = self._validar_puntos(puntos)

    def _validar_puntos(self, puntos):
        try:
            puntos_int = int(puntos)          # falla si es "hola" o "5.5"
        except (ValueError, TypeError):
            raise ValueError(f"Puntos inválidos: '{puntos}'. Deben ser entero.")
        if puntos_int < self.PUNTOS_MIN:
            raise ValueError(f"Los puntos no pueden ser negativos: {puntos_int}.")
        if puntos_int > self.PUNTOS_MAX:
            raise ValueError(
                f"Puntos fuera de rango: {puntos_int}. Máximo: {self.PUNTOS_MAX:,}."
            )
        return puntos_int
```

salida de errores en consola — todos manejados con try/except en main.py

--- Admin: nivel inválido ---

Admin("Test", "t@x.com", "abc")

✗ Error: Nivel inválido: 'abc'. Debe ser un número entero.

Admin("Test", "t@x.com", "3.5")

✗ Error: Nivel inválido: '3.5'. Debe ser un número entero.

Admin("Test", "t@x.com", 6)

✗ Error: Fuera de rango: 6. Debe estar entre 1 y 5.

--- Cliente: puntos inválidos ---

Cliente("Test", "t@x.com", "hola")

✗ Error: Puntos inválidos: 'hola'. Deben ser entero.

Cliente("Test", "t@x.com", -10)

✗ Error: Los puntos no pueden ser negativos: -10.

Cliente("Test", "t@x.com", 200000)

✗ Error: Puntos fuera de rango: 200000. Máximo: 100,000.

--- Casos válidos ---

Admin("Carlos", "carlos@admin.com", 5) ✓ nivel_acceso = 5

Admin("Luis", "luis@admin.com", "3") ✓ nivel_acceso = 3 (string → int)

Cliente("Ana", "ana@mail.com", "350") ✓ puntos = 350 (string → int)

6 Captura de Ejecución

python3 main.py

LISTADO DE USUARIOS DEL SISTEMA

► Usuario #1 (Admin)

¡Hola, Carlos Ramírez! Bienvenido al sistema.

Nombre : Carlos Ramírez

Email : carlos@admin.com

Nivel de acceso: 5

[Carlos Ramírez] ADMINISTRADOR – Acceso TOTAL. Nivel: 5

► Usuario #2 (Cliente)

¡Hola, Ana Torres! Bienvenido al sistema.

Nombre : Ana Torres

Email : ana@cliente.com

Puntos acumulados: 350

[Ana Torres] CLIENTE – Acceso al catálogo y perfil. Puntos: 350

► Usuario #3 (Invitado)

¡Hola, Luis Gómez! Bienvenido al sistema.

Nombre : Luis Gómez

Email : luis@correo.com

[Luis Gómez] INVITADO – Solo lectura. Acceso limitado.

SISTEMA DE USUARIOS v1.0

1. Mostrar todos los usuarios
2. Agregar Administrador
3. Agregar Cliente
4. Agregar Invitado
5. Salir

Elige una opción: _

7 Herencia, Sobrescritura y super()



Herencia

Permite que `Admin`, `Cliente` e `Invitado` obtengan automáticamente los atributos y métodos de `Usuario` sin repetir código. Se declara con `class` `Admin(Usuario)`.



Sobrescritura

Cada clase hija redefine `acceso_sistema()` con su propio comportamiento. Python usa el método de la clase más específica. Esto permite el **polimorfismo**.



super()

Llama al constructor o método de la clase padre. En `Admin.__init__`, `super().__init__(nombre, email)` ejecuta primero la lógica de `Usuario` antes de agregar `nivel_acceso`.

8 Conclusiones

¿Qué aprendí?

Que la herencia permite estructurar el código de forma lógica y reutilizable. Definir una vez en la clase base y extender según sea necesario es mucho más limpio que repetir código.

¿Qué fue lo más difícil?

Entender cuándo usar `super()` correctamente, especialmente al sobrescribir métodos que también deben llamar a la versión del padre como `mostrar_datos()`.

¿Qué ventajas tiene la herencia?

Evita duplicar código, facilita el mantenimiento (un cambio en la clase base se propaga a todas las hijas) y permite el polimorfismo para tratar distintos tipos de forma uniforme.

¿Dónde se usa en la vida real?

En sistemas de gestión de usuarios (e-commerce, redes sociales), frameworks web, videojuegos (tipos de personajes), y cualquier sistema con roles y permisos diferenciados.

Preguntas

a.¿Qué ventajas ofrece la herencia en comparación con repetir código?

Con herencia, los atributos nombre y email y los métodos mostrar_datos(), saludar() y _validar_email() se definen una sola vez en Usuario y las tres clases hijas los obtienen automáticamente. Sin herencia, ese bloque de código tendría que copiarse en Admin, Cliente e Invitado por separado.

Esto implica tres ventajas directas: si se necesita cambiar la validación del email, el cambio se hace en un solo lugar y se propaga a todas las clases; el código es más corto y fácil de leer; y hay menos posibilidad de que una clase quede desactualizada respecto a las demás.

b.¿Qué métodos fueron heredados?

Las tres clases hijas heredan de Usuario los siguientes métodos sin necesidad de redefinirlos:

saludar()_validar_email()mostrar_datos() — base

Adicionalmente, Admin y Cliente extienden mostrar_datos() usando super() para agregar sus atributos propios, sin perder el comportamiento original. acceso_sistema() también se hereda, pero las tres clases lo sobrescriben, por lo que en la práctica siempre se ejecuta la versión propia de cada clase.

c.¿Por qué sobrescribir acceso_sistema()?

Porque cada tipo de usuario tiene permisos distintos y no tiene sentido que todos respondan igual. El método en Usuario es genérico y sirve como punto de partida, pero:

CON SOBRESCRITURA

Cada clase define exactamente qué puede hacer su tipo de usuario. Al recorrer una lista mixta, Python llama automáticamente al método correcto según el tipo real del objeto — eso es polimorfismo.

SIN SOBRESCRITURA

Todos mostrarían el mismo mensaje genérico de acceso, sin importar si son administradores con acceso total o invitados con solo lectura. El sistema perdería su diferenciación.

d.¿Qué diferencia existe entre un atributo heredado y uno propio?

Un atributo heredado existe en la clase padre y todas las hijas lo reciben automáticamente. Un atributo propio solo existe en esa clase específica y se define dentro de su propio `__init__`, después de llamar a `super()`.

Atributo	Definido en	Disponible en	Tipo
nombre	Usuario	Admin, Cliente, Invitado	heredado
email	Usuario	Admin, Cliente, Invitado	heredado
nivel_acceso	Admin	Solo Admin	propio
puntos	Cliente	Solo Cliente	propio

e.¿Qué pasaría si todas las clases estuvieran separadas y no heredaran?

Cada clase tendría que definir por su cuenta los atributos nombre y email, la validación de email, el método `saludar()` y `mostrar_datos()`. Ese bloque se repetiría idéntico en Admin, Cliente e Invitado.

PROBLEMA DE MANTENIMIENTO

Un cambio en la lógica de validación de email obligaría a modificar los tres archivos por separado, con riesgo de que queden inconsistentes.

PROBLEMA DE POLIMORFISMO

Sin una clase base común no es posible recorrer una lista de usuarios mixtos y llamar a `acceso_sistema()` de forma uniforme — habría que tratar cada tipo por separado con condiciones.

En resumen, el sistema funcionaría pero sería frágil, difícil de escalar y muy propenso a errores al agregar un nuevo tipo de usuario en el futuro.